

Sistema de Gestión para Tienda de Pinturas

1. Introducción

Para mi proyecto final he desarrollado una solución integral de escritorio orientada a la gestión de una **Tienda de Pinturas**. Mi objetivo principal con este proyecto ha sido crear una aplicación robusta, segura y con una interfaz (Dashboard) que permita a distintos tipos de usuarios interactuar con el sistema según su rol (empleados y clientes). El sistema permite el registro de nuevos usuarios, el control exhaustivo del inventario de pinturas y la gestión automatizada de las transacciones de compra.

2. Objetivos del Proyecto

Durante el desarrollo del sistema, me propuse alcanzar los siguientes objetivos:

- **Gestión Segura de Usuarios:** Implementar un sistema de autenticación y registro (Login/Registro) que diferencie de manera clara entre clientes y empleados, asignando permisos específicos según el rol.
- **Gestión de Inventario:** Desarrollar un módulo CRUD (Crear, Leer, Actualizar, Borrar) para el control total de los productos de pintura.
- **Control de Accesos:** Restringir las funcionalidades operativas: asegurar que los clientes solo puedan consultar el catálogo y realizar compras, bloqueando su capacidad de crear o eliminar productos.
- **Automatización de Compras:** Crear un flujo de compras inteligente que calcule de manera automática el precio total basado en la cantidad solicitada.
- **Actualización de Stock en Tiempo Real:** Integrar disparadores a nivel de código para que el stock de una pintura se descuente automáticamente de la base de datos cada vez que un cliente confirma una compra.

3. Tecnologías y Entorno de Desarrollo

Para la construcción del sistema, he elegido el siguiente stack tecnológico:

- **Lenguaje de Programación:** Java (JDK 8 o superior).
- **Interfaz Gráfica:** Java Swing, personalizando el aspecto visual mediante el uso del tema *Nimbus* para una interfaz más moderna.
- **Base de Datos:** MySQL. El servidor de base de datos se ejecuta mediante un entorno contenedorizado en **Docker**, mapeado al puerto 3307 para evitar conflictos con instalaciones locales.
- **Conexión a Datos:** Controlador JDBC de MySQL (mysql-connector-java-8.0.30.jar).
- **Control de Versiones:** Git, con un repositorio remoto alojado en GitHub.

4. Arquitectura del Sistema y Patrones de Diseño

He desarrollado el sistema aplicando una arquitectura **MVC (Model-View-Controller)** en conjunto con los patrones **DAO (Data Access Object)** y **DTO (Data Transfer Object)**. Esto me ha permitido asegurar que la interfaz gráfica esté completamente desacoplada de la lógica de negocio y de persistencia.

El proyecto está estructurado en los siguientes paquetes:

- **db/:** Contiene la clase `ConexionDB.java`, donde centralizo y configuro la conexión a la base de datos MySQL aplicando el patrón *Singleton*.
- **model/:** Contiene las clases POJO (*Plain Old Java Object*) que representan las entidades principales de mi dominio de negocio (Usuario, Cliente, Pintura, Compra, Empleado).
- **dao/:** En este paquete he definido las interfaces y sus respectivas implementaciones (ej. `UsuarioDAO`, `PinturaDAOImpl`). Aquí encapsulo todas las consultas y sentencias SQL, utilizando `PreparedStatement` para prevenir ataques de Inyección SQL.
- **dto/:** He implementado el patrón DTO creando clases como `CompraDTO`, esenciales para agrupar y transferir datos complejos que provienen de consultas con múltiples cruces de tablas (JOIN). Esto evita ensuciar las vistas con lógica de base de datos.
- **view/:** Agrupa todas las interfaces gráficas de usuario diseñadas en Swing (Login, Registro, Principal).

5. Diseño y Modelo de Base de Datos

He diseñado un modelo relacional estructurado, aplicando el concepto de **Joined Table Inheritance** (Herencia mediante tablas unidas) para modelar de manera eficiente la jerarquía de los usuarios del sistema.

- **Tabla usuarios:** Es la entidad base. Almacena los datos comunes: username, password, email, nombre, apellidos, dni y rol.
- **Tabla clientes:** Entidad hija conectada a usuarios mediante `usuario_id` como clave foránea y primaria. Añade el campo `tipo_cliente` (minorista/mayorista). He aplicado la restricción `ON DELETE CASCADE` para mantener la integridad.
- **Tabla empleados:** Entidad hija de usuarios que añade los atributos turno y salario. También cuenta con eliminación en cascada.
- **Tabla pinturas:** Entidad principal del inventario. Almacena la información del producto: nombre, color, tipo, precio y stock.
- **Tabla compras:** Tabla intermedia que resuelve la relación Muchos-a-Muchos (N:M) entre *clientes* y *pinturas*. Actúa como un registro de transacciones, almacenando la fecha de compra, cantidad adquirida y el coste total.

6. Funcionalidades Clave y Soluciones Técnicas

6.1. Transacciones Atómicas en Registro

Para el proceso de registro de nuevos clientes y empleados (que involucra insertar primero en usuarios y luego en las tablas hijas), he abandonado el comportamiento por defecto de `auto-commit`. En su lugar, gestiono las transacciones manualmente usando `con.setAutoCommit(false)`, `con.commit()` y `con.rollback()`. De esta forma garantizo la atomicidad: si falla la inserción en la tabla hija, se revierte la creación del usuario base, manteniendo la base de datos íntegra y sin registros huérfanos.

6.2. Separación de Lógica por Roles

La vista Principal detecta dinámicamente qué tipo de usuario ha iniciado sesión. Si el usuario logueado es un **Cliente**, el sistema oculta o deshabilita los botones de "Crear producto", "Editar" y "Eliminar". De esta manera, garantizo que los clientes solo interactúen con el catálogo y procedan a comprar.

6.3. Automatización de Compras y Control de Stock

En el panel de compras, he automatizado el cálculo matemático para la comodidad del usuario. Una vez que se selecciona una pintura y una cantidad, el sistema calcula el coste total de la operación cruzando el precio unitario del producto con la demanda solicitada. Además, he codificado que, al confirmar la transacción de compra, no solo se guarde el registro en la tabla compras, sino que inmediatamente lance una consulta UPDATE que resta la cantidad comprada al stock actual de dicha pintura, manteniendo siempre actualizado el inventario.

7. Despliegue y Ejecución

Para la correcta ejecución e inicialización del proyecto, es necesario:

1. **Entorno de Base de datos:** Levantar un contenedor Docker de MySQL expuesto en el puerto 3307. Posteriormente, ejecutar el script `database.sql` (ubicado en el directorio raíz) que se encarga de estructurar el esquema `tiendapinturas` y rellenarlo con datos semilla de prueba.
2. **Configuración del Proyecto IDE:** Importar el código a un entorno de desarrollo (Eclipse, IntelliJ IDEA, VS Code), y asegurar que el driver `mysql-connector-java-8.0.30.jar` (ubicado en `lib/`) esté añadido al *Build Path* del proyecto.
3. **Lanzamiento:** Ejecutar la clase principal `Main.java` que configura la apariencia inicial y despliega la pantalla de Login.

8. Conclusiones y Futuras Líneas de Mejora

El desarrollo de la "Tienda de Pinturas" ha culminado en un software funcional que cumple enteramente con las especificaciones iniciales y garantiza una interacción fluida con la base de datos. La implementación de MVC y DAO proporciona un código limpio, ordenado y altamente escalable. Como futuras implementaciones, valoro añadir un módulo de facturación automatizada que genere un PDF (recibo) tras cada compra, y la encriptación asimétrica de las contraseñas de usuario en base de datos.